	PCET'S Pimpri Chinchwad College of Engineering, Pune

Name: Soham Kulkarni **PRN**: 124B1F056

Course: Design and Analysis of Algorithm Laboratory

ASSIGNMENT NO. 03

---

1. Problem Statement

A devastating flood has hit multiple villages in a remote area, and the government, along with NGOs, is organizing an emergency relief operation. A rescue team has a limited-capacity boat that can carry a maximum weight of W kilograms. The boat must transport critical supplies, including food, medicine, and drinking water, from a relief center to the affected villages.

Each type of relief item has:

- A weight (wi) in kilograms.
- Utility value (vi) indicating its importance (e.g., medicine has higher value than food).

Some items can be divided into smaller portions (e.g., food and water), while others must be taken as a whole (e.g., medical kits).

2. Goals

- Implement the Fractional Knapsack algorithm to maximize the total utility value of the
- supplies transported.
- Prioritize high-value items while considering weight constraints.
- Allow partial selection of divisible items (e.g., carrying a fraction of food packets). ▪ Ensure that the boat carries the most critical supplies given its weight limit W.

3. Course Objectives

▪ To know the basics of computational complexity of various algorithms. ▪ To select appropriate algorithm design strategies to solve real-world problems.

4. Course Outcomes

After learning the course, students will be able to:

- Analyze the asymptotic performance of algorithms.
- Solve computational problems by applying suitable paradigms such as Greedy method.

5. Theory

The Fractional Knapsack Problem is a classic optimization problem: given a set of items— each with weight wi and value vi—and a maximum capacity W, the objective is to maximize total value by selecting items (or fractions of them) up to capacity. Unlike the 0/1 Knapsack, the fractional version allows partial selection, making it solvable optimally in polynomial time.

Algorithm (Greedy Strategy): Compute the value-to-weight ratio vi / wi for each item. Sort items in descending order of this ratio.

Fill the knapsack: Take items fully if they fit. If capacity runs out, take the exact fraction needed of the current item (only if divisible). Skip indivisible items that do not fit.

5.1. Working of Greedy:

1. Greedy-choice property: Choosing the highest ratio item at each step is always part of an optimal solution.

2. Optimal substructure: Once part of the knapsack is filled, the remaining capacity forms a smaller instance of the same problem.

5.2. Time Complexity

- Ratio computation: O(n)
- Sorting: O(n log n)
- Selection: O(n)
- Total: O(n log n)

5.3. Pseudo Code

```
function MaxUtilFractional(items, W):
        Input: items = list of (weight w, value v, isDivisible), capacity W
        Output: maximum total utility value

        Compute ratio r[i] = v[i] / w[i] for each item
        Sort items by descending r[i]

        totalValue = 0
        remainingCapacity = W

        for each item in sorted items:
                if remainingCapacity == 0:
                        break
                if item.weight <= remainingCapacity:
```

```
                totalValue += item.value
                remainingCapacity -= item.weight
            else if item.isDivisible:
                fraction = remainingCapacity / item.weight
                totalValue += fraction * item.value
                remainingCapacity = 0
                // else if indivisible and can't fit, skip
return totalValue
```

Sample Example:

Maximum Utility = 100 (if all fit in given W).

Boat capacity: 17 kg

Step 1: Calculate utility per kg

- Medicine: 50 / 5 = 10
- Food: 30 / 10 = 3
- Water: 20 / 20 = 1

Preference order: Medicine → Food → Water

Step 2: Fill the boat

- Take Medicine fully:
  - Weight used = 5 kg
  - Utility = 50
  - Remaining capacity = 17 - 5 = 12 kg
- Take Food fully (weight 10 kg, divisible):
  - Fits completely

- Utility = 30
- Remaining capacity = 12 - 10 = 2 kg

▪ Take Water partially (weight 20 kg, divisible):

- Only 2 kg fits
- Utility = 20 × (2/20) = 2

Step 3: Total Utility

● Medicine: 50

● Food: 30

● Water (partial): 2

Total Utility = 82

6. OUTPUT

Github Linkhttps://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

7. Conclusion

The Fractional Knapsack Algorithm maximizes utility by prioritizing items with the highest value-to-weight ratio. In emergency relief logistics, this ensures that life-saving items like medicine kits are transported first, followed by food and water as space allows. The greedy algorithm is both efficient (O(n log n)) and optimal for fractional cases, making it highly effective in disaster management where quick, resource-optimized decisions are critical.